ZIPLOC SAS

Plataforma de billeteras digitales, recompensas y analítica de transacciones

INFORME TÉCNICO FINAL

Presentado por
Juan Esteban Galeano Bolaños
Juan Steban Acosta Ramírez

Asignatura: Estructuras de Datos
Universidad del Quindío
Noviembre de 2026

Resumen

ZIPLOC SAS es una aplicación académica que simula una plataforma de billeteras digitales. Integra administración de usuarios y billeteras, movimientos financieros, operaciones programadas, fidelización mediante puntos, notificaciones y analítica. Su propósito formativo es mostrar cómo distintas estructuras de datos resuelven problemas concretos de organización, búsqueda, priorización, consulta y trazabilidad.

La solución utiliza un backend desarrollado con Java y Spring Boot, persistencia documental en MongoDB y una interfaz web construida con React, Vite y Tailwind CSS. La lógica se organiza en capas: controladores REST, servicios de aplicación, modelos de dominio, estructuras de datos e infraestructura de persistencia.

El proyecto implementa una lista enlazada y una pila para el historial y la reversión de transacciones; una cola circular para notificaciones; un heap mínimo para operaciones programadas; un árbol AVL para ordenar usuarios por puntos; tablas hash para localizar usuarios y billeteras; y grafos dirigidos y ponderados para representar transferencias. También incorpora validación recursiva de integridad, analítica y categorías de gasto.

Este informe describe el diseño y la implementación identificados en el código y la documentación del proyecto. La compilación y las pruebas integrales deben ejecutarse en el entorno local de entrega; por tanto, no se presentan como aprobadas si no existe evidencia de su ejecución.

Tabla de contenido

- 1. Introducción
- 2. Planteamiento del problema
- 3. Objetivos
- 4. Alcance y requisitos
- 5. Tecnologías utilizadas
- 6. Arquitectura del sistema
- 7. Modelo de dominio
- 8. Implementación de estructuras de datos
- 9. Lógica de negocio y algoritmos
- 10. API y módulos funcionales
- 11. Interfaz de usuario

- 12. Persistencia, seguridad y manejo de errores
- 13. Pruebas y validación
- 14. Análisis de complejidad
- 15. Limitaciones y mejoras futuras
- 16. Conclusiones
- 17. Referencias técnicas

1. Introducción

Las billeteras digitales permiten administrar fondos y realizar operaciones financieras desde una aplicación. Para que estos sistemas sean útiles, deben mantener saldos consistentes, registrar movimientos, permitir consultas y responder de forma eficiente ante solicitudes concurrentes o repetitivas. La estructura interna de los datos influye directamente en el costo de estas operaciones.

ZIPLOC SAS aborda este escenario mediante un simulador académico. El proyecto no busca operar como una entidad financiera real ni procesar dinero bancario; reproduce procesos habituales de una billetera virtual para estudiar la aplicación de estructuras de datos y algoritmos en un dominio reconocible.

El desarrollo integra funcionalidades básicas —recargas, retiros, transferencias, consultas e historial— con componentes avanzados de recompensas, programación de operaciones, notificaciones, grafos y analítica.

2. Planteamiento del problema

Una aplicación de billetera debe administrar múltiples usuarios, varias billeteras por usuario y un historial creciente de transacciones. Si todas las operaciones se resolvieran mediante recorridos lineales, las consultas y actualizaciones podrían volverse costosas a medida que aumenta el volumen de información. Asimismo, las operaciones futuras requieren un mecanismo que seleccione primero las que ya deben ejecutarse, mientras que las notificaciones necesitan conservar un conjunto reciente sin crecer indefinidamente.

El problema académico consiste en seleccionar e implementar estructuras adecuadas para cada necesidad, justificar su complejidad y conectarlas con reglas de negocio que mantengan la coherencia del sistema.

3. Objetivos

3.1 Objetivo general

Desarrollar una plataforma web de simulación de billeteras digitales que aplique estructuras de datos y algoritmos para gestionar usuarios, billeteras, transacciones, recompensas, operaciones programadas, notificaciones y análisis de actividad financiera.

3.2 Objetivos específicos

- Implementar operaciones de recarga, retiro, transferencia, consulta de saldo e historial.

- Aplicar listas y pilas para registrar movimientos y soportar la reversión de operaciones.
- Utilizar una cola circular para administrar notificaciones recientes con capacidad limitada.
- Implementar una cola de prioridad basada en heap para ordenar operaciones por fecha y prioridad.
- Organizar usuarios por puntos mediante un árbol AVL y un índice auxiliar por identificador.
- Emplear tablas hash para localizar usuarios y billeteras de manera eficiente.
- Representar relaciones de transferencia mediante grafos dirigidos y ponderados.
- Calcular puntos, actualizar niveles y permitir el canje de beneficios.
- Generar reportes y análisis de patrones de gasto, y verificar recursivamente la integridad del historial.

4. Alcance y requisitos

4.1 Funcionalidades básicas

Requisito	Descripción dentro de ZIPLOC SAS
Usuarios	Registro, consulta, actualización, eliminación y autenticación.
Billeteras	Creación y administración de múltiples billeteras asociadas a un usuario.
Recargas	Acreditación de fondos en una billetera.
Retiros	Débito sujeto a validación de saldo disponible.
Transferencias	Movimiento entre billeteras, con registro del historial y relaciones del grafo.
Consultas	Consulta de saldo e historial de movimientos.

4.2 Funcionalidades avanzadas

Requisito	Implementación identificada
Puntos y beneficios	SistemaRecompensas calcula puntos por tipo y monto, actualiza usuarios y permite canjes.
Niveles	Clasificación Bronce, Plata, Oro y Platino según los puntos acumulados.

Operaciones programadas	Operaciones futuras y recurrentes ordenadas por fecha y prioridad.
Patrones de gasto	ServicioAnalitica agrupa egresos por categoriaGasto; los registros históricos sin categoría usan una clasificación de compatibilidad basada en la billetera origen.
Alertas	Notificaciones recientes administradas mediante cola circular; se contempla el recordatorio de operación programada.
Multimonedero	Un usuario puede administrar distintas billeteras; el grafo de billeteras representa movimientos entre ellas.
Integridad	Verificación recursiva del historial de transacciones.
Interfaz	Aplicación web React con vistas para los módulos funcionales.

El alcance es académico. La existencia de endpoints o clases no sustituye las pruebas de aceptación de cada flujo ni constituye una certificación de seguridad o de operación financiera real.

5. Tecnologías utilizadas

Componente	Tecnología / herramienta	Responsabilidad
Lenguaje backend	Java 21 (configuración del pom.xml)	Implementación del dominio, servicios y estructuras.
Framework backend	Spring Boot 4.0.5, Spring Web	Exposición de API REST e inyección de dependencias.
Persistencia	Spring Data MongoDB / MongoDB	Almacenamiento de documentos de usuarios, billeteras, transacciones, alertas y operaciones.
Construcción backend	Maven Wrapper	Gestión de dependencias,

		compilación y pruebas.
Frontend	React 18 y Vite	Interfaz de usuario y desarrollo del cliente web.
Estilos	Tailwind CSS	Diseño y estilos de la interfaz.
HTTP	Axios	Comunicación entre frontend y backend.
Pruebas	Spring Boot Starter Test / JUnit	Pruebas unitarias y de flujo.

Nota de configuración: el archivo pom.xml del proyecto declara Java 21 y Spring Boot 4.0.5. Debe utilizarse un JDK compatible con esa configuración. La documentación de ejecución debe mantenerse sincronizada con el JDK realmente instalado en el equipo.

6. Arquitectura del sistema

La aplicación sigue una arquitectura por capas que separa la recepción de solicitudes, las reglas de negocio, el dominio y la persistencia.

Capa	Paquete / componentes	Función
Presentación / API	api.controller, api.dto	Recibe solicitudes HTTP, valida el formato de entrada y construye respuestas.
Aplicación	application.service	Coordina reglas de negocio, validaciones y estructuras de datos.
Dominio	domain.model, domain.enums	Representa usuarios, billeteras, transacciones, operaciones, alertas, beneficios y estados.
Estructuras	domain.structures	Implementa las estructuras de datos requeridas por el proyecto.
Infraestructura	infrastructure.persistence, repository, scheduler	Integra MongoDB, repositorios y ejecución

		automática de operaciones.

6.1 Flujo general de una solicitud

Cliente web → Axios → Controller REST → Servicio de aplicación → Dominio / estructuras → Repositorio MongoDB → Respuesta HTTP

Los controladores exponen endpoints y delegan la lógica a los servicios. Los servicios aplican las reglas de negocio y utilizan el contexto de plataforma para coordinar estructuras compartidas. Los repositorios permiten guardar y recuperar información persistente.

6.2 Flujo de una transferencia

1. El cliente envía una solicitud de transferencia al endpoint correspondiente.
2. TransaccionController recibe el DTO y delega la operación a TransaccionService.
3. Se validan las billeteras involucradas, su estado y la disponibilidad de saldo.
4. BilleteraService actualiza los saldos de origen y destino.
5. GestorTransacciones registra el movimiento y lo incorpora a la pila de reversión.
6. GrafoTransferencias registra la relación dirigida y ponderada entre los nodos correspondientes.
7. SistemaRecompensas calcula los puntos y actualiza el nivel del usuario según las reglas configuradas.
8. Los servicios de riesgo, auditoría y notificaciones registran la información complementaria cuando corresponde.
9. El controlador devuelve la respuesta de la API.

7. Modelo de dominio

Entidad	Responsabilidad
Usuario	Representa al cliente; contiene datos de identificación, estado, puntos y nivel.
Billetera	Representa un monedero asociado a un usuario, con tipo, estado y saldo.
Transaccion	Registra tipo, valor, fecha, estado, billeteras involucradas y datos complementarios como puntos y categoría de gasto.

OperacionProgramada	Representa una operación futura o recurrente, su fecha, prioridad, frecuencia, estado e información de ejecución.
Alerta	Representa una notificación asociada a un usuario y a un tipo de evento.
Beneficio	Define una recompensa canjeable, sus puntos requeridos, nivel y tipo.
EventoAuditoria	Registra eventos relevantes para trazabilidad y análisis.

Las enumeraciones del dominio centralizan valores permitidos, por ejemplo tipos y estados de transacción, niveles de usuario, tipos de billetera, recurrencias, niveles de riesgo y tipos de alerta.

8. Implementación de estructuras de datos

8.1 Lista enlazada — historial de transacciones

La clase GestorTransacciones mantiene nodos enlazados. Cada nodo contiene una transacción y una referencia al siguiente nodo. El método agregar inserta al inicio de la lista, por lo que la inserción es O(1). El recorrido del historial y la búsqueda por identificador requieren recorrer nodos, por lo que su costo es O(n).

La inserción al inicio permite mantener un historial en el que la transacción más reciente queda primero. La estructura evita desplazar elementos como ocurriría en un arreglo al insertar al comienzo.

8.2 Pila — reversión de transacciones

GestorTransacciones utiliza una pila para conservar las transacciones reversibles. La pila sigue el principio LIFO: la última transacción incorporada es la primera que se obtiene para revertir. Las operaciones push y pop son O(1) en condiciones normales.

La pila identifica el movimiento más reciente, mientras que la lógica de TransaccionService debe aplicar las compensaciones necesarias sobre saldos, estados y puntos. La reversión no debe entenderse como borrar el registro: se conserva la trazabilidad del movimiento y se registra su estado de reversión.

8.3 Cola circular — notificaciones

ColaNotificaciones utiliza un arreglo de capacidad fija y los índices front y rear. El avance de los índices se realiza mediante módulo de la capacidad, de manera que al llegar al final del arreglo se continúa desde la primera posición. Cuando está llena, la estructura desplaza lógicamente la alerta más antigua antes de insertar la nueva.

Encolar y despachar tienen costo O(1). Consultar el historial reciente requiere O(k), donde k es la cantidad de alertas almacenadas. La implementación valida que la capacidad sea mayor que cero para evitar operaciones de módulo inválidas.

8.4 Heap mínimo — operaciones programadas

ColaProgramadas implementa una cola de prioridad mediante un heap binario mínimo almacenado en una lista dinámica. La comparación ordena primero por fecha de ejecución y después por prioridad numérica. La operación más próxima queda en la raíz.

Insertar y extraer la raíz requieren O(log n), mientras que consultar la raíz con peek requiere O(1). La estructura permite procesar operaciones vencidas sin ordenar toda la colección en cada consulta. El servicio reconstruye el heap a partir de las operaciones pendientes persistidas para recuperar el estado después de reiniciar la aplicación.

8.5 Árbol AVL — fidelización

ArbolFidelizacion mantiene usuarios ordenados por puntos acumulados y utiliza el identificador como criterio de desempate. Como árbol AVL, actualiza alturas y aplica rotaciones LL, RR, LR y RL para conservar el balance.

Las búsquedas, inserciones y eliminaciones en el árbol balanceado tienen costo O(log n). El código incorpora un índice hash auxiliar por ID, que permite localizar el nodo de un usuario en O(1) promedio. Las consultas por rango pueden expresarse como O(log n + k), donde k es la cantidad de resultados.

8.6 Tablas hash — usuarios y billeteras

TablaUsuarios utiliza encadenamiento separado para resolver colisiones. Cada posición puede contener una cadena de elementos que comparten índice hash. TablaBilleteras utiliza sondeo lineal: ante una colisión, busca la siguiente posición disponible siguiendo la tabla.

La búsqueda, inserción y actualización presentan costo promedio O(1) bajo una distribución adecuada de claves y un factor de carga controlado. En el peor caso pueden degradarse a O(n). Estas estructuras favorecen consultas frecuentes por identificador.

8.7 Grafo dirigido y ponderado — transferencias

GrafoTransferencias representa las relaciones entre usuarios o billeteras mediante nodos y aristas dirigidas. El peso de una arista representa el monto asociado al movimiento. La dirección permite distinguir origen y destino.

El grafo permite consultar vecinos y relaciones, sumar montos entre nodos y analizar conectividad. El recorrido de vecinos depende del grado del nodo cuando se utiliza una representación por listas de adyacencia. La plataforma también contempla un grafo de movimientos entre billeteras.

8.8 Resumen de complejidades

Estructura	Operación	Complejidad esperada
Lista enlazada	Insertar al inicio	O(1)
Lista enlazada	Buscar / recorrer	O(n)
Pila	Apilar / desapilar	O(1)
Cola circular	Encolar / despachar	O(1)
Cola circular	Consultar alertas recientes	O(k)
Heap mínimo	Insertar / extraer mínimo	O(log n)
Heap mínimo	Consultar mínimo	O(1)
Árbol AVL	Buscar / insertar / eliminar	O(log n)
Índice hash auxiliar	Buscar por ID	O(1) promedio
Tabla hash	Buscar / insertar	O(1) promedio; O(n) peor caso
Grafo	Recorrer vecinos	O(grado del nodo)

9. Lógica de negocio y algoritmos

9.1 Sistema de puntos

SistemaRecompensas calcula los puntos mediante bloques completos de cien unidades monetarias. La implementación identificada aplica las siguientes reglas:

Tipo de movimiento	Regla implementada
RECARGA	1 punto por cada bloque completo de 100 unidades.
RETIRO	2 puntos por cada bloque completo de 100 unidades.
TRANSFERENCIA	3 puntos por cada bloque completo de 100 unidades.
PAGO_PROGRAMADO	3 puntos por cada bloque completo de 100 unidades más 10 puntos.

La fórmula utiliza floor(valor / 100). Por ejemplo, una transferencia de 1.000 unidades produce 30 puntos; un retiro de 500 unidades produce 10 puntos. En pagos programados, un monto de 1.000 unidades produce 40 puntos bajo la regla actual.

El servicio aplica los puntos al usuario y almacena los puntos generados en la transacción. Cuando se revierte una operación, la lógica de reversión debe compensar los puntos asociados para mantener la consistencia.

9.2 Niveles y beneficios

El dominio contempla los niveles Bronce, Plata, Oro y Platino. El servicio de recompensas registra beneficios base y valida que el usuario cumpla el nivel requerido y disponga de los puntos suficientes antes del canje. Al confirmar el canje, descuenta los puntos, registra el beneficio y genera una notificación.

Los umbrales concretos de nivel deben consultarse en la enumeración o regla de clasificación vigente del proyecto y mantenerse iguales en backend, frontend e informe. No se deben confundir los puntos requeridos por un beneficio con los límites de puntos de cada nivel.

9.3 Operaciones programadas y recurrencia

OperacionProgramadaService valida que la operación tenga valor positivo, fecha y tipo definidos, y billeteras válidas. Asigna identificador, estado inicial y usuario asociado; persiste la operación y la incorpora al heap. El sistema contempla recurrencias sin repetición, diaria, semanal y mensual.

El procesamiento selecciona operaciones vencidas con respecto a una fecha de corte. Las operaciones fallidas conservan información del error y pueden quedar pendientes para un intento

posterior. La clave operacionProgramadaId permite relacionar el movimiento financiero con la programación que lo originó y apoyar el control de duplicados.

9.4 Análisis de patrones de gasto

La analítica agrupa egresos por categoría de gasto. Se incorporó el atributo categoriaGasto a la transacción y a su DTO, de modo que los nuevos movimientos puedan clasificarse en categorías como alimentación, transporte, vivienda, servicios, salud, educación, entretenimiento, compras u otros.

Para mantener compatibilidad con documentos históricos que no tengan categoriaGasto, el análisis utiliza como alternativa el tipo de billetera de origen. Esta alternativa permite incluir registros antiguos, pero no equivale a una categoría comercial detallada.

9.5 Verificación recursiva de integridad

El servicio de transacciones expone una operación de verificación de integridad recursiva. Su finalidad es recorrer el historial y comprobar las condiciones de consistencia definidas por el dominio, devolviendo un resultado con el estado general y los detalles de posibles inconsistencias. El endpoint documentado en el controlador es GET /api/transacciones/integridad.

9.6 Auditoría y riesgo

El sistema incorpora eventos de auditoría y elementos de análisis de riesgo para conservar trazabilidad de actividades. La analítica permite consultar eventos asociados a un usuario y generar resúmenes de actividad. Estos mecanismos son parte de la simulación académica y no equivalen a un motor antifraude certificado.

10. API y módulos funcionales

La API utiliza el contexto base /api. Las rutas específicas se deben contrastar con los controladores actuales al preparar una entrega, pues la documentación puede quedar desactualizada cuando se modifican endpoints.

Módulo	Rutas representativas	Operaciones
Usuarios	/api/usuarios	Registro, consulta, actualización, eliminación, login y ranking.
Billeteras	/api/billeteras	Creación, consulta,

		actualización, eliminación y saldo.
Transacciones	/api/transacciones	Creación, recarga, retiro, transferencia, consulta, historial, reversión e integridad.
Programadas	/api/programadas	Crear, listar pendientes/procesadas y procesar operaciones vencidas.
Recompensas	/api/recompensas	Consultar puntos y nivel, listar beneficios y canjear.
Notificaciones	/api/notificaciones	Consultar alertas, despachar y marcar como leídas.
Analítica	/api/analitica	Reportes, frecuencia, categorías activas, patrones de gasto, grafos, auditoría y rendimiento.

Ejemplos de rutas de analítica presentes en AnaliticaController: GET /api/analitica/reporte/{usuarioId}, /top-billeteras, /usuarios-activos, /auditoria/{usuarioId}, /frecuencia-tipos, /categorias-activas, /patrones-gasto, /monto-total, /grafo/{usuarioId}, /grafo-billeteras, /transacciones-mayor-valor y /rendimiento/{usuarioId}.

11. Interfaz de usuario

El frontend está desarrollado con React y Vite, utiliza Axios para consumir la API y Tailwind CSS para la presentación visual. La estructura del cliente separa páginas, componentes reutilizables, hooks de consulta, servicios HTTP, autenticación y estilos.

Entre las vistas funcionales documentadas se encuentran Dashboard, Usuarios, Billeteras, Transacciones, Analítica, Recompensas, Notificaciones y Operaciones programadas. La interfaz ofrece una experiencia centralizada para administrar el monedero y consultar la actividad.

La comunicación se realiza mediante servicios y hooks. React Query administra el estado de las consultas, el almacenamiento temporal de resultados y la actualización de información remota.

12. Persistencia, seguridad y manejo de errores

12.1 Persistencia

MongoDB se utiliza mediante Spring Data MongoDB y repositorios específicos. La persistencia conserva información entre reinicios, mientras que las estructuras propias en memoria facilitan la demostración de algoritmos y operaciones. Por ello, no debe describirse el sistema como exclusivamente en memoria: el proyecto combina estructuras en memoria con persistencia documental.

12.2 Seguridad de acceso

El frontend y backend contemplan mecanismos de autenticación y protección de rutas. El backend contiene servicios de credenciales y control de acceso. La configuración exacta de tokens, expiración y permisos debe documentarse según el archivo de configuración activo y no se presume aquí una auditoría de seguridad.

12.3 Errores y validaciones

Los controladores utilizan DTO para separar los datos de entrada y salida del modelo de dominio. ApiExceptionHandler centraliza el tratamiento de excepciones de la API. Los servicios validan condiciones de negocio, como saldo suficiente, valores positivos, existencia de billeteras, fechas y tipos de operación.

13. Pruebas y validación

El proyecto contiene pruebas de servicios y pruebas de flujo, entre ellas pruebas de usuario, del árbol de fidelización y de operaciones programadas. La presencia de clases de prueba demuestra que se definieron escenarios automatizados, pero no prueba por sí sola que todas las pruebas pasen en el equipo de entrega.

13.1 Comandos de verificación

Desde la raíz del backend, con un JDK compatible con Java 21 y MongoDB disponible cuando las pruebas lo requieran:

```
mvnw.cmd clean test
```

Para el frontend, desde la carpeta ziplocSAS_Front:

```
npm install
npm run build
```

13.2 Casos de prueba recomendados

- Recargar una billetera y verificar que el saldo aumente por el valor exacto.
- Intentar retirar un monto superior al saldo y comprobar que la operación sea rechazada.
- Transferir entre dos billeteras y comprobar ambos saldos, el historial y el grafo.
- Revertir una transacción y verificar saldo, estado, puntos y trazabilidad.
- Registrar puntos para distintos tipos y montos, incluidos valores inferiores a cien.
- Canjear un beneficio con puntos suficientes y rechazar un canje sin saldo de puntos o nivel requerido.
- Programar operaciones con fechas distintas y comprobar el orden del heap.
- Comprobar que una operación futura no se ejecute antes de su fecha y que una vencida sí se procese.
- Verificar la generación del recordatorio RECORDATORIO_PROGRAMADA.
- Insertar alertas hasta superar la capacidad de la cola circular y confirmar que se descarte la más antigua.
- Crear transacciones con distintas categoriasGasto y consultar el reporte de patrones.
- Ejecutar la verificación recursiva con un historial consistente y con un caso de inconsistencia controlado.

Estado de verificación de este informe: no se ejecutaron los comandos de compilación ni pruebas en el entorno de generación del documento. Por tanto, la validación final debe realizarse localmente y registrarse con los resultados reales de Maven y Vite.

14. Análisis de complejidad y decisiones

La selección de estructuras responde a la operación dominante de cada módulo. La lista enlazada facilita inserciones al inicio del historial; la pila permite obtener la última transacción reversible; la cola circular limita el almacenamiento de notificaciones; el heap prioriza operaciones por fecha; el AVL conserva orden y balance en la clasificación; las tablas hash aceleran búsquedas por clave; y el grafo modela relaciones de transferencia.

Las complejidades promedio de las tablas hash dependen de una función hash adecuada y de la distribución de claves. En escenarios de colisiones intensas, su rendimiento puede degradarse. Del mismo modo, las consultas sobre grafos dependen de la representación interna y del número de conexiones.

El uso de MongoDB junto con estructuras en memoria responde a dos objetivos diferentes: persistir el estado de negocio y demostrar el comportamiento algorítmico de las estructuras requeridas.

15. Limitaciones y mejoras futuras

- Ejecutar y documentar una batería completa de pruebas automatizadas y de integración en el entorno definitivo.
- Revisar transacciones concurrentes y atomicidad de las actualizaciones de saldo para escenarios de múltiples solicitudes simultáneas.
- Definir políticas explícitas de precisión monetaria; para sistemas financieros reales se recomienda evitar double y utilizar BigDecimal con reglas de redondeo documentadas.
- Fortalecer la idempotencia y el control de reintentos de operaciones programadas, verificando los casos de fallo parcial.
- Ampliar los reportes con filtros por periodo, usuario y categoría, y documentar la semántica exacta de cada indicador.
- Agregar pruebas de carga para comparar las estructuras con volúmenes crecientes de datos.
- Documentar los umbrales de niveles y reglas de puntos en un único lugar para evitar divergencias entre backend, frontend e informe.
- Revisar la configuración de seguridad, CORS, variables de entorno y credenciales antes de desplegar fuera del entorno académico.

16. Conclusiones

ZIPLOC SAS integra los componentes principales solicitados para un simulador de monedero virtual con sistema de puntos. El proyecto relaciona cada estructura de datos con una necesidad funcional: historial, reversión, notificaciones, programación, clasificación, búsqueda y análisis de transferencias.

La implementación del heap mínimo permite seleccionar operaciones programadas según fecha y prioridad; el AVL mantiene ordenada la clasificación de usuarios por puntos; las tablas hash apoyan búsquedas por identificador; y el grafo dirigido representa el flujo de transferencias. La lista

enlazada, la pila y la cola circular completan los mecanismos de administración de movimientos y alertas.

La combinación de Spring Boot, MongoDB y React ofrece una separación clara entre interfaz, lógica de negocio y persistencia. Además, los módulos de analítica, auditoría y recompensas amplían el ejercicio más allá de las operaciones básicas.

Como conclusión académica, el valor principal del proyecto está en demostrar que la selección de una estructura de datos debe responder al tipo de operación que se desea optimizar. Para cerrar la entrega es indispensable ejecutar las pruebas en el equipo, corregir cualquier fallo detectado y conservar evidencia de los resultados.

17. Referencias técnicas

- Documento de requisitos: Proyecto Final — Simulador de un Monedero Virtual con sistema de Puntos, asignatura Estructuras de Datos.
- Código fuente del proyecto ZIPLOC SAS: backend Java/Spring Boot, frontend React/Vite y configuración Maven.
- Documentación oficial de Java: https://docs.oracle.com/en/java/
- Documentación oficial de Spring Boot: https://docs.spring.io/spring-boot/
- Documentación oficial de Spring Data MongoDB: https://docs.spring.io/spring-data/mongodb/reference/
- Documentación oficial de MongoDB: https://www.mongodb.com/docs/
- Documentación oficial de React: https://react.dev/
- Documentación oficial de Vite: https://vite.dev/guide/